

CS 305: Computer Networks

Fall 2025

Network Layer – The Control Plane

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- ❖ link state (global)
- ❖ distance vector (decentralized)

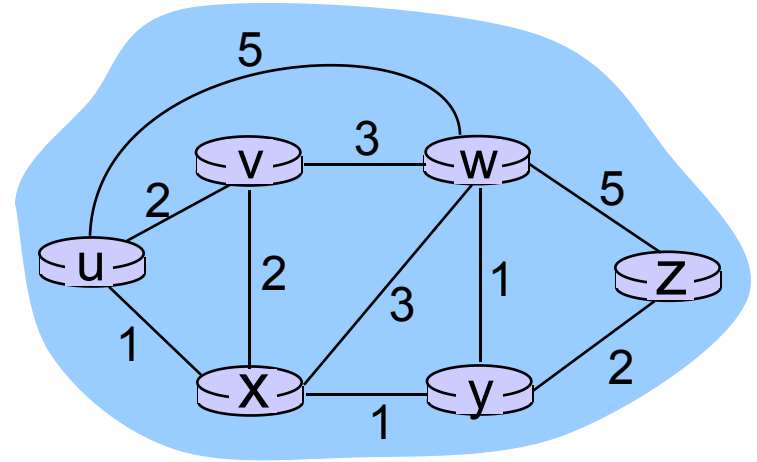
5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP



A link-state routing algorithm

Dijkstra's algorithm

- ❖ net topology, link costs known to all nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- ❖ computes least cost paths from one node (‘source’) to all other nodes
 - gives *forwarding table* for that node
- ❖ iterative: after k iterations, know least cost path to k dest.’s

notation:

- ❖ $c(x,y)$: link cost from node x to y; $= \infty$ if not direct neighbors
- ❖ $D(v)$: current value of cost of path from source to dest. v
- ❖ $p(v)$: predecessor node along path from source to v
- ❖ N' : set of nodes whose least cost path definitively known

Dijkstra's algorithm

1 **Initialization:**

2 $N' = \{u\}$

3 for all nodes v

4 if v adjacent to u

5 then $D(v) = c(u, v)$

6 else $D(v) = \infty$

7

8 **Loop**

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min(D(v), D(w) + c(w, v))$**

13 /* new cost to v is either old cost to v or known

14 shortest path cost to w plus cost from w to v */

15 **until all nodes in N'**

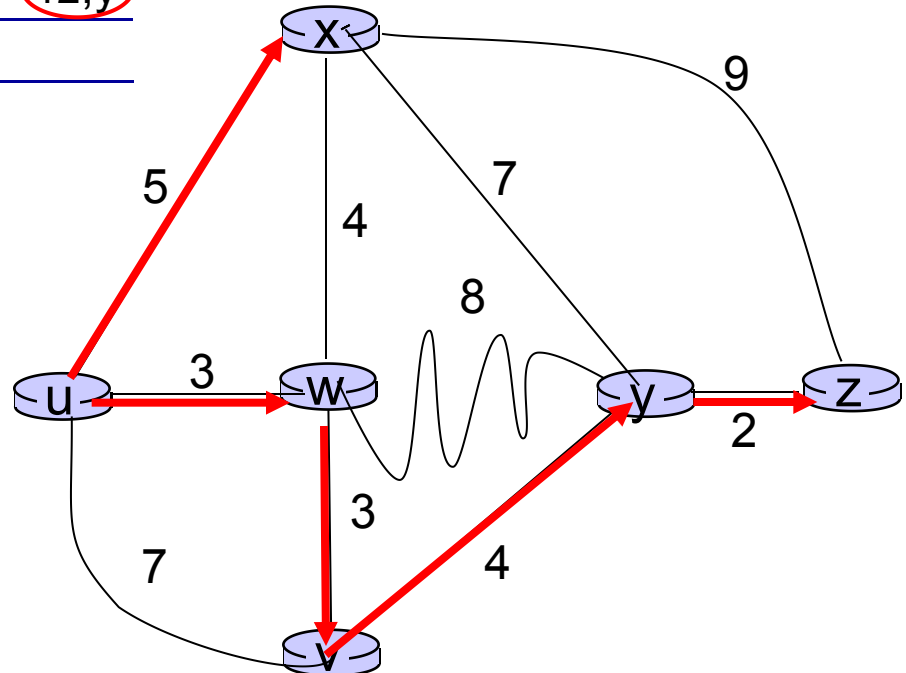


Dijkstra's algorithm: example

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

notes:

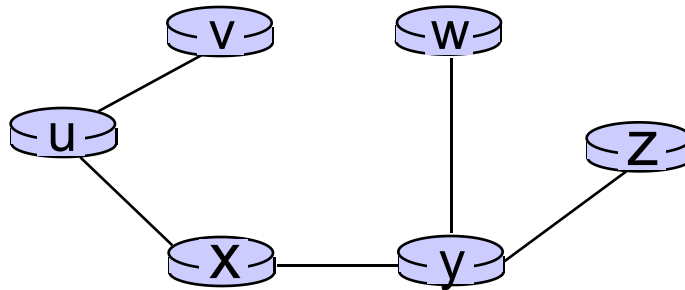
- ❖ construct shortest path tree by tracing predecessor nodes
- ❖ ties can exist (can be broken arbitrarily)



Dijkstra's algorithm: example

(2)

resulting shortest-path tree from u:



resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

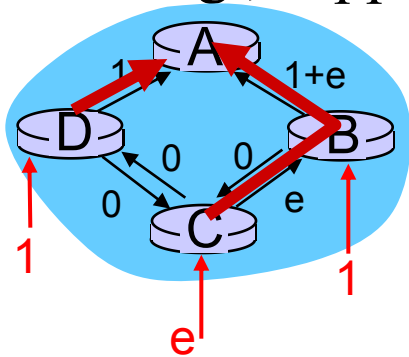
Dijkstra's algorithm, discussion

algorithm complexity: n nodes

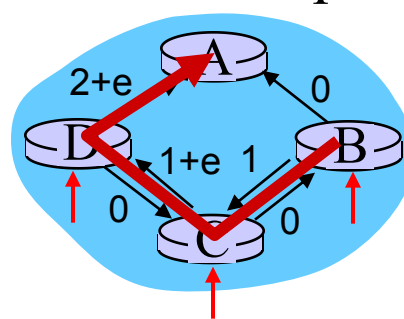
- ❖ each iteration: need to check all nodes not in N'
- ❖ $n(n+1)/2$ comparisons: $O(n^2)$
- ❖ more efficient implementations possible: $O(n \log n)$

Oscillations (摇摆) possible (if we consider congestion) :

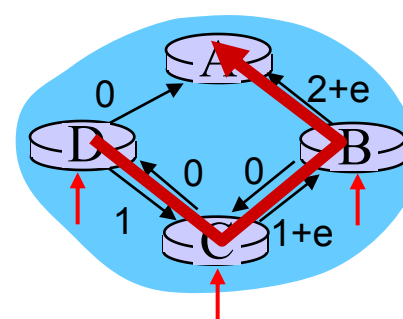
- ❖ e.g., support link cost equals amount of carried traffic:



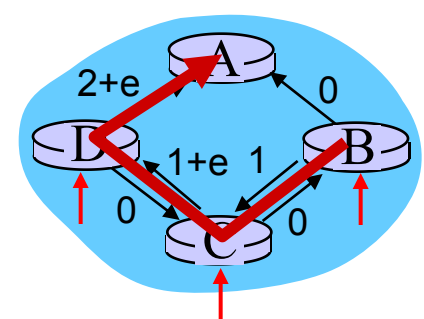
initially



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- ❖ link state (global)
- ❖ distance vector (decentralized)

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

Distance vector algorithm

The distance-vector (DV) algorithm:

- **distributed**: each node **receives** some information from one or more of its directly attached neighbors, performs a **calculation**, and then **distributes** the results of its calculation back to its neighbors.
- **Iterative**: this process continues on until no more information is exchanged between neighbors.
- **Asynchronous**: it does not require all of the nodes to operate in lockstep with each other.

Bellman-Ford equation

Distance vector algorithm

Distance vector algorithm

Bellman-Ford equation:

$d_x(y) :=$ cost of least-cost path from x to y

then

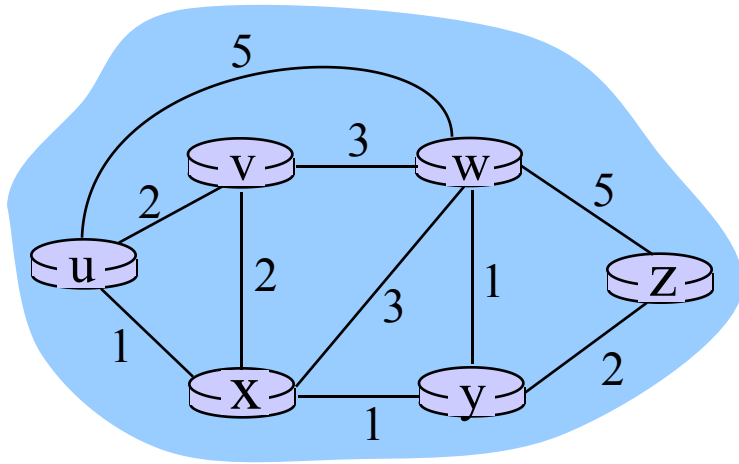
$$d_x(y) = \min_v \{ c(x,v) + d_v(y) \}$$

cost from neighbor v to destination y

cost to neighbor v

$\min$ taken over all neighbors v of x

Bellman-Ford example



clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

B-F equation says:

$$\begin{aligned} d_u(z) &= \min \{ c(u,v) + d_v(z), \\ &\quad c(u,x) + d_x(z), \\ &\quad c(u,w) + d_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

Node achieving minimum is

- the next hop in shortest path used in forwarding table

Distance vector algorithm

$D_x(y)$ = estimate of least cost from x to y

Node x:

- knows cost to each neighbor v: $c(x,v)$
- maintains its recent distance vector $\mathbf{D}_x = [D_x(y): y \in N]$
- maintains its neighbors' recent distance vectors. For each neighbor v, x maintains $\mathbf{D}_v = [D_v(y): y \in N]$

Distance vector algorithm

Key Idea:

- From time-to-time, each node sends its own recent distance vector (DV) to neighbors
- When x receives new DV from neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\} \text{ for each node } y \in N$$

- If its DV has changed, sends the updated DV to neighbors
...

- ❖ under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

Distance vector algorithm

iterative, asynchronous:

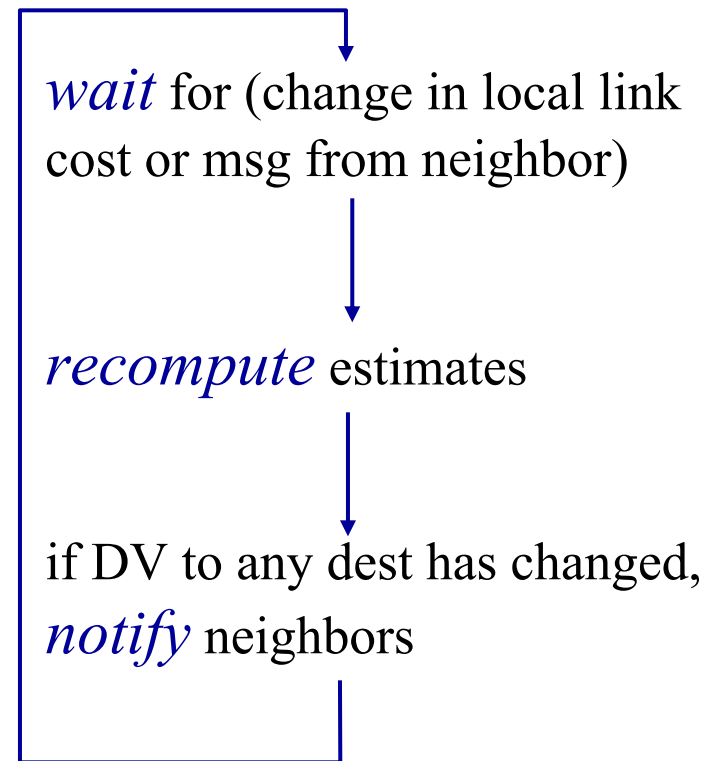
each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed:

- each node notifies neighbors *only* when its DV changes
 - neighbors then notify their neighbors if necessary

each node:



Distance vector algorithm

```
1  Initialization:
2    for all destinations  $y$  in  $N$ :
3       $D_x(y) = c(x, y)$  /* if  $y$  is not a neighbor then  $c(x, y) = \infty$  */
4    for each neighbor  $w$ 
5       $D_w(y) = ?$  for all destinations  $y$  in  $N$ 
6    for each neighbor  $w$ 
7      send distance vector  $\mathbf{D}_x = [D_x(y) : y \text{ in } N]$  to  $w$ 
8
9  loop
10    wait (until I see a link cost change to some neighbor  $w$  or
11           until I receive a distance vector from some neighbor  $w$ )
12
13    for each  $y$  in  $N$ :
14       $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$ 
15
16    if  $D_x(y)$  changed for any destination  $y$ 
17      send distance vector  $\mathbf{D}_x = [D_x(y) : y \text{ in } N]$  to all neighbors
18
19  forever
```

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

**node x
table**

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

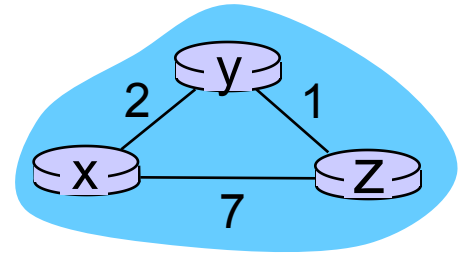
		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

**node y
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

**node z
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0



time

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

**node x
table**

	cost to		
	x	y	z
from x	0	2	7
from y	∞	∞	∞
from z	∞	∞	∞

**node y
table**

	cost to		
	x	y	z
from x	∞	∞	∞
from y	2	0	1
from z	∞	∞	∞

**node z
table**

	cost to		
	x	y	z
from x	∞	∞	∞
from y	∞	∞	∞
from z	7	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	7	1	0

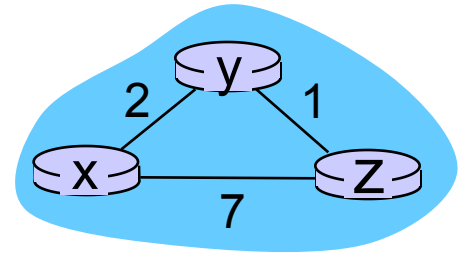
	cost to		
	x	y	z
from x	0	2	7
from y	2	0	1
from z	7	1	0

	cost to		
	x	y	z
from x	0	2	7
from y	2	0	1
from z	3	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	3	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	3	1	0

	cost to		
	x	y	z
from x	0	2	3
from y	2	0	1
from z	3	1	0

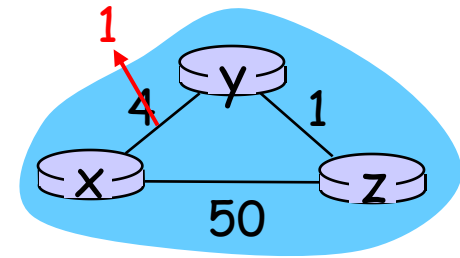


time →

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ updates routing info, recalculates distance vector
- ❖ if DV changes, notify neighbors



“good
news
travels
fast”

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

t_1 : z receives update from y , updates its table, computes new least cost to x , sends its neighbors its DV.

t_2 : y receives z 's update, updates its distance table. y 's least costs do *not* change, so y does *not* send a message to z .

Distance vector: link cost changes

node x
table

		cost to		
		x	y	z
from	x	0	4 1	5 2
	y	4	0	1
	z	5	1	0

Detect $c(x,y)=c(y,x)=1$!

node y
table

		cost to		
		x	y	z
from	x	0	4	5
	y	4 1	0	1
	z	5	1	0

node z
table

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	5	1	0

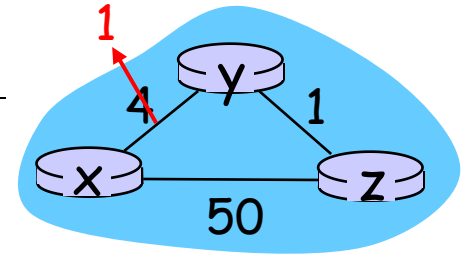
		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0



$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

“good news travels fast”

time

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$$

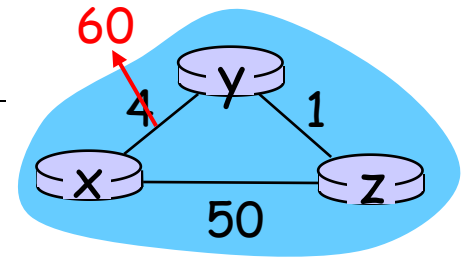
$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$$

**node x
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



Detect $c(x,y) = c(y,x) = 60$!

**node y
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	6	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	8	0	1
	z	7	1	0

**node z
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	6	0	1
	z	7	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

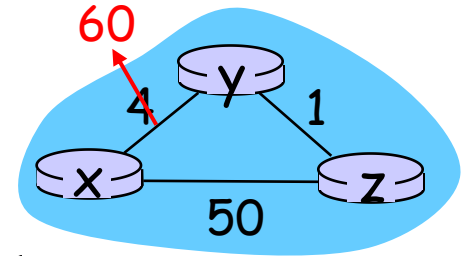
time

loop will persist for 44 iterations until z eventually computes the cost of its path via y to be greater than 50.

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ *bad news travels slow* - “count to infinity” problem!
- ❖ 44 iterations before algorithm stabilizes:
 - ❖ $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60 + 0, 1 + 5\} = 6$
 - ❖ $D_z(x) = \min\{c(z,x) + D_x(x), c(z,y) + D_y(x)\} = \min\{50 + 0, 1 + 6\} = 7$
 - ❖ $D_y(x) = 8, D_z(x) = 9, \dots$ totally 44 iteration!



Poisoned reverse:

- ❖ If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❖ will this completely solve count to infinity problem?

Without Poisoned Reverse

**node x
table**

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

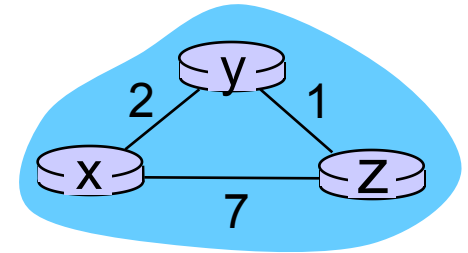
		cost to		
		x	y	z
from	x			
	y			
	z			

**node y
table**

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



**node z
table**

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

time

With Poisoned Reverse

**node x
table**

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

**node y
table**

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			

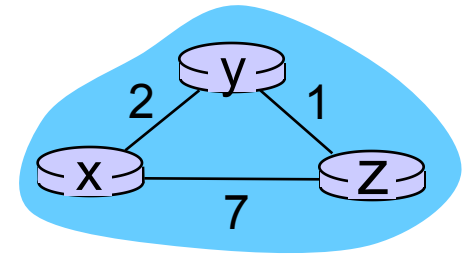
		cost to		
		x	y	z
from	x			
	y			
	z			

**node z
table**

		cost to		
		x	y	z
from	x			
	y			
	z			

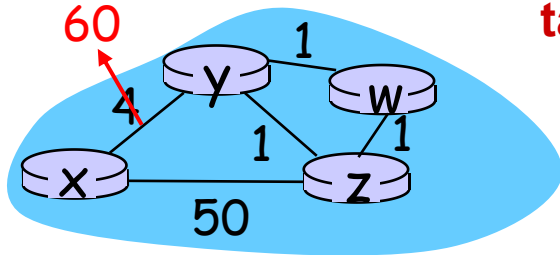
		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



time

Distance vector: link cost changes



node x table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	4	0	1	1	
z	5	1	0	1	

node y table

	cost to				
	x	y	z	w	
from	x	0	4	∞	∞
y	4	0	1	1	
z	∞	1	0	1	
w	∞	1	1	0	

node z table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	4	0	1	1	
z	5	1	0	1	
w	5	1	1	0	

node z table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	60	0	1	1	
z	6	1	0	1	
w	5	1	1	0	

node w table

	cost to				
	x	y	z	w	
from	y	60	0	1	1
z	5	1	0	1	
w	6	1	1	0	

node w table

	cost to				
	x	y	z	w	
from	y	4	0	1	1
z	5	1	0	1	
w	5	1	1	0	

node y table

	cost to				
	x	y	z	w	
from	x	0	4	∞	∞
y	7	0	1	1	
z	6	1	0	1	
w	6	1	1	0	

Poisoned reverse:

- ❖ will this completely solve count to infinity problem?
- ❖ No, when the loops involves three or more nodes

Comparison of LS and DV algorithms

message complexity

- **LS:** with n nodes, E links, $O(nE)$ msgs sent
- **DV:** exchange between neighbors only
 - convergence time varies

speed of convergence

- **LS:** $O(n^2)$ algorithm requires $O(nE)$ msgs
- **DV:** convergence time varies
 - may be routing loops
 - count-to-infinity problem

robustness: what happens if router malfunctions?

LS:

- node can advertise incorrect *link* cost
- each node computes only its *own* table

DV:

- DV node can advertise incorrect *path* cost
- each node's table used by others
 - error propagate thru network

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

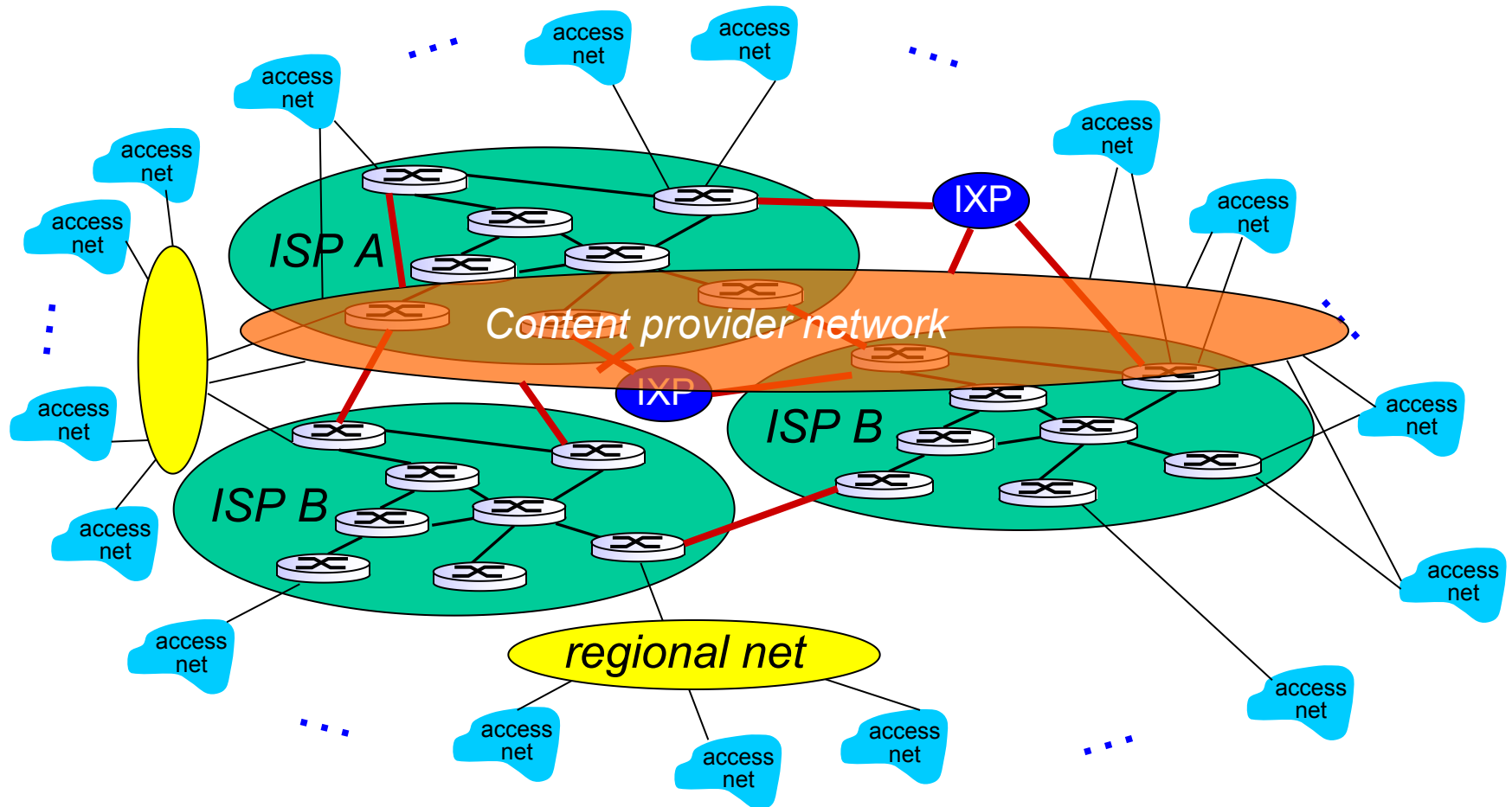
Making routing scalable

The link state and distance vector routing studies far is idealized

- all routers identical
- network “flat”

... *not* true in practice

Review the Architecture of Internet



The link state routing doesn't work on the Internet!

Making routing scalable

The link state and distance vector routing studies far is idealized

- all routers identical
- network “flat”

... *not* true in practice

scale: with billions of destinations:

- can't store all destinations in routing tables!
- routing table exchange would swamp links!

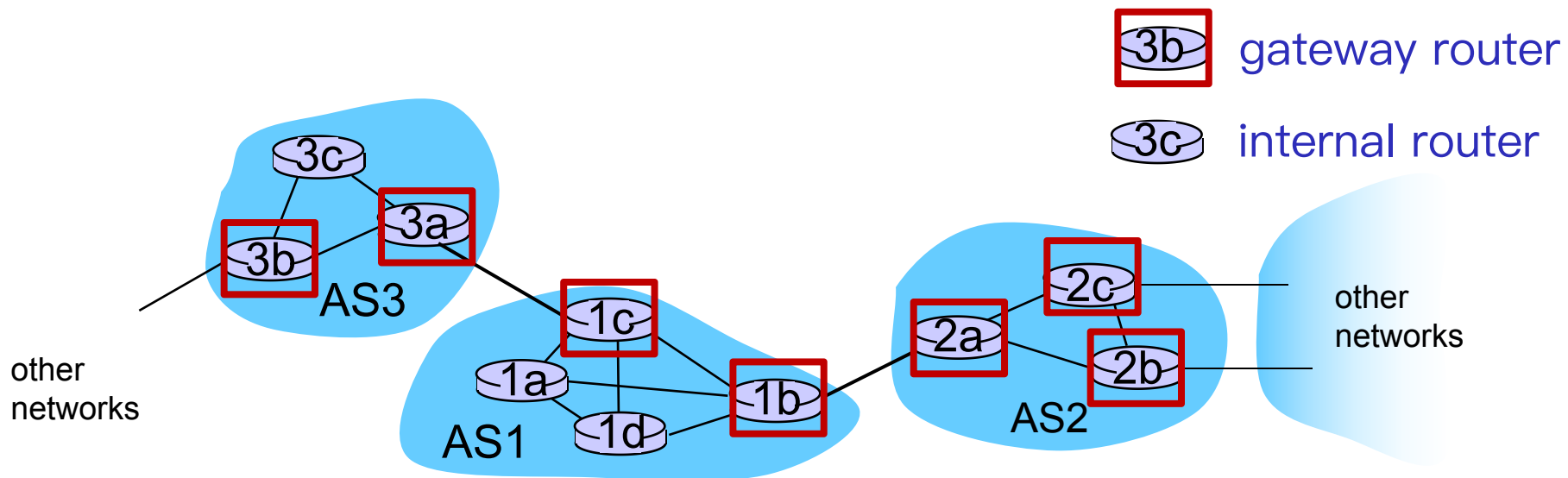
administrative autonomy

- Internet = network of networks
- each network admin may want to control routing in its own network

Internet approach to scalable routing

aggregate routers into regions known as “autonomous systems” (AS) (a.k.a. “domains”)

- Gateway router: at “edge” of its own AS, has link(s) to router(s) in other AS
- Interior router: no link to other AS



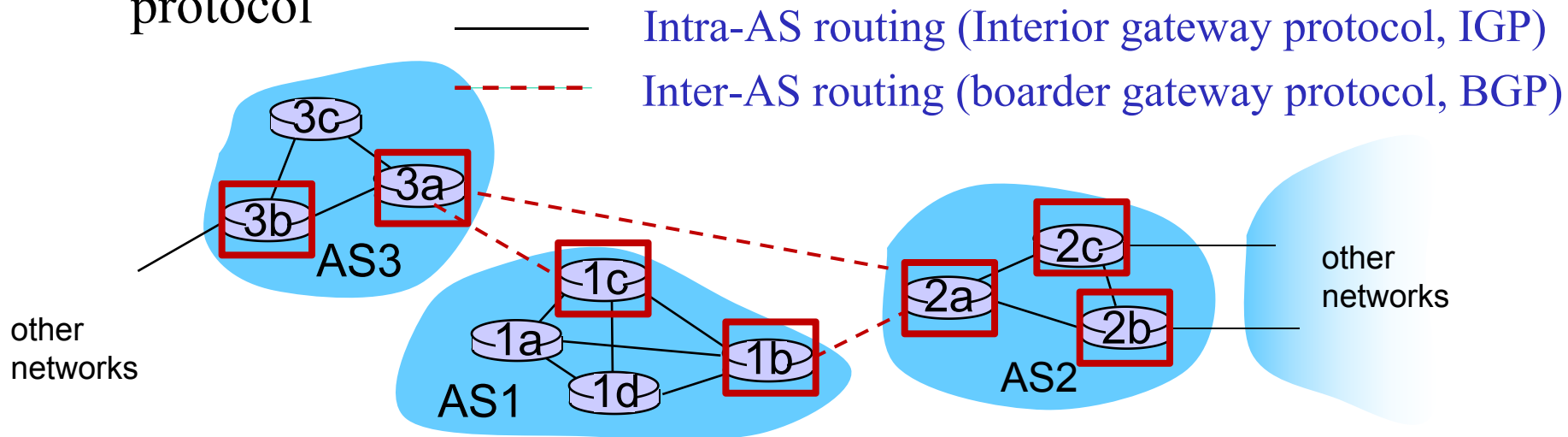
Internet approach to scalable routing

intra-AS routing

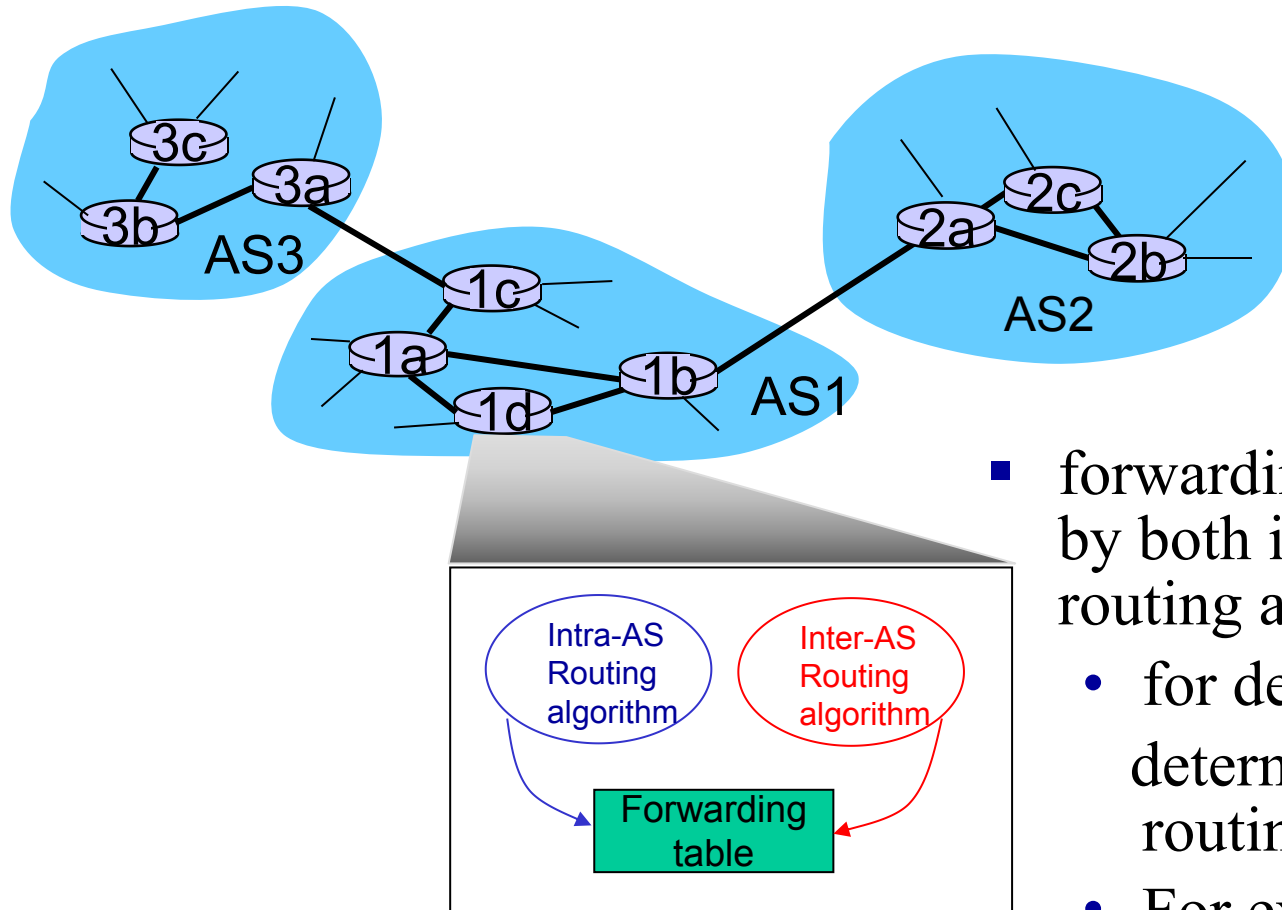
- routing among hosts, routers in same AS (“network”)
- all routers in AS must run *same* intra-domain protocol
- routers in *different* AS can run *different* intra-AS routing protocol

inter-AS routing

- routing among AS'es
- gateways perform inter-AS routing (as well as intra-AS routing)



Interconnected ASes



- forwarding table configured by both intra- and inter-AS routing algorithm
 - for destinations within AS: determined by intra-AS routing
 - For external destinations: determined by both inter-AS & intra-AS routing

Intra-AS Routing

- also known as *interior gateway protocols (IGP)*
- most common intra-AS routing protocols:
 - RIP: Routing Information Protocol (distance vector-based)
 - OSPF: Open Shortest Path First (link state-based)
 - IS-IS protocol essentially same as OSPF
 - IGRP: Interior Gateway Routing Protocol (Cisco proprietary for decades, until 2016)

Inter-AS tasks

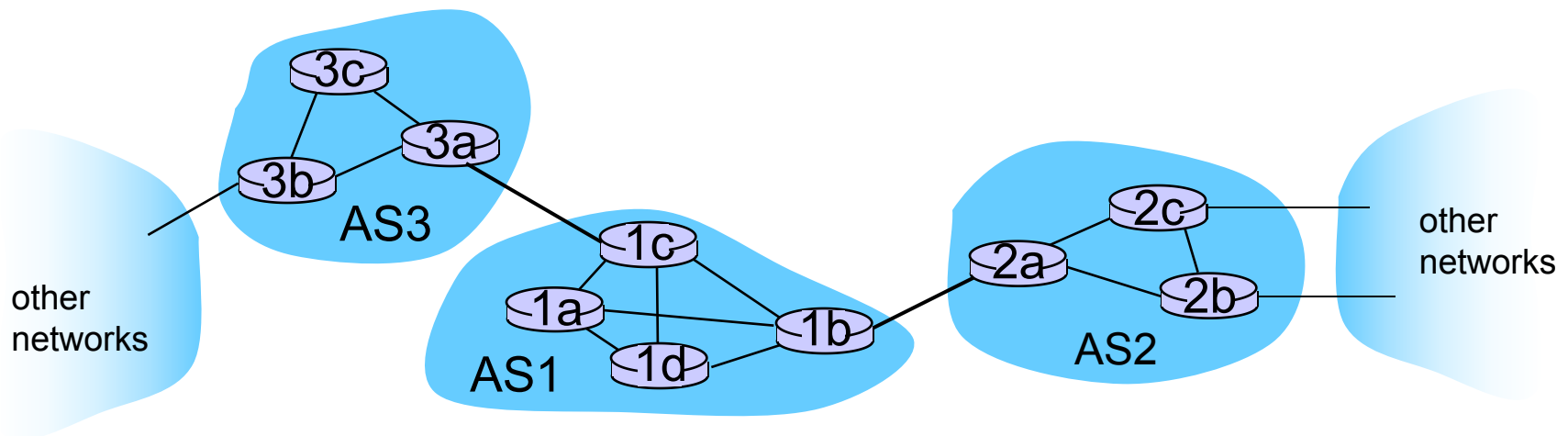
- suppose router in AS1 receives datagram destined outside of AS1:
 - router should forward packet to gateway router, but which one?

Border gateway protocols (BGP)

AS1 must:

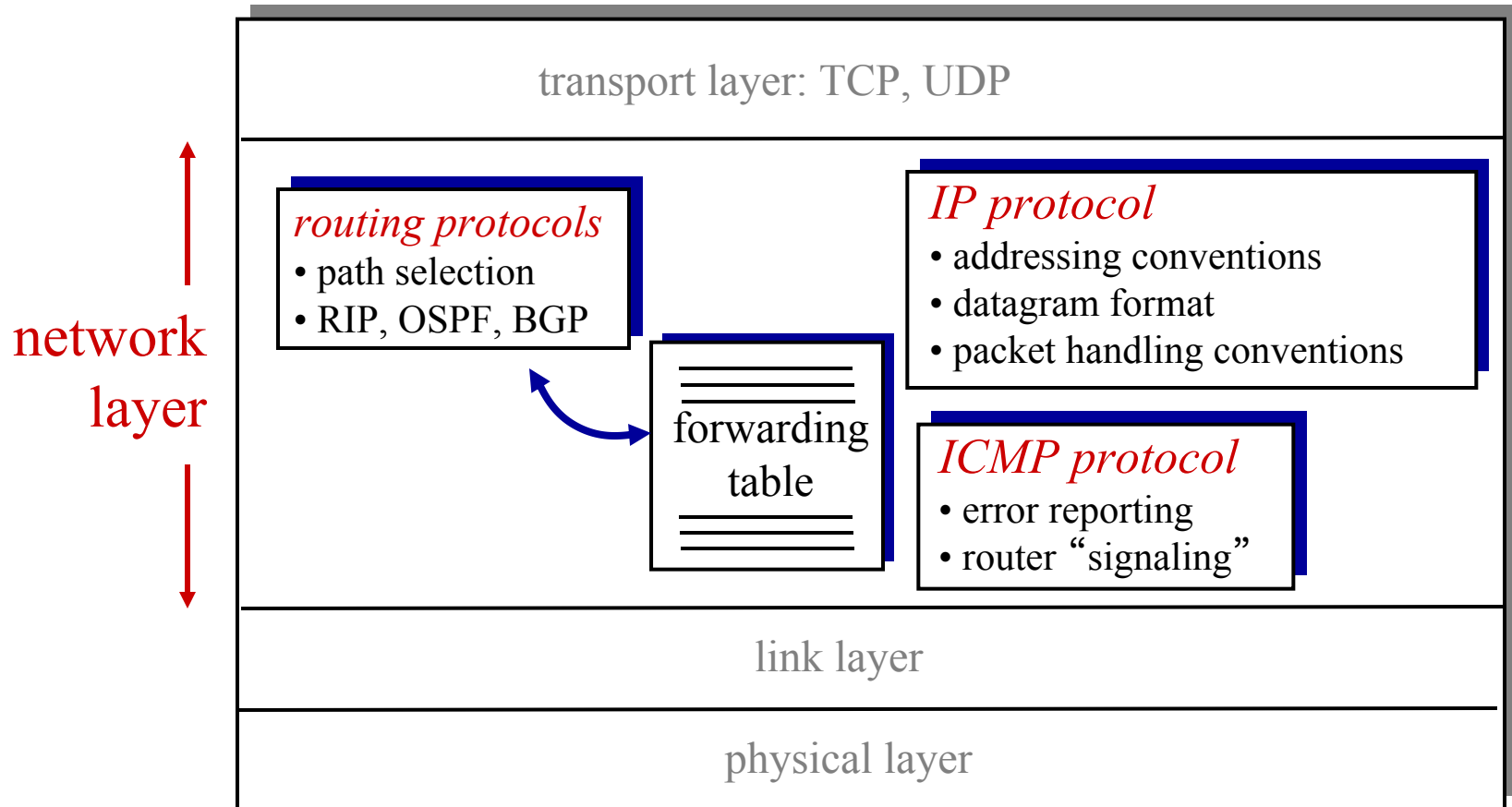
1. learn which dests are reachable through AS2, which through AS3
2. propagate this reachability info to all routers in AS1

job of inter-AS routing!



The Internet network layer

host, router network layer functions:



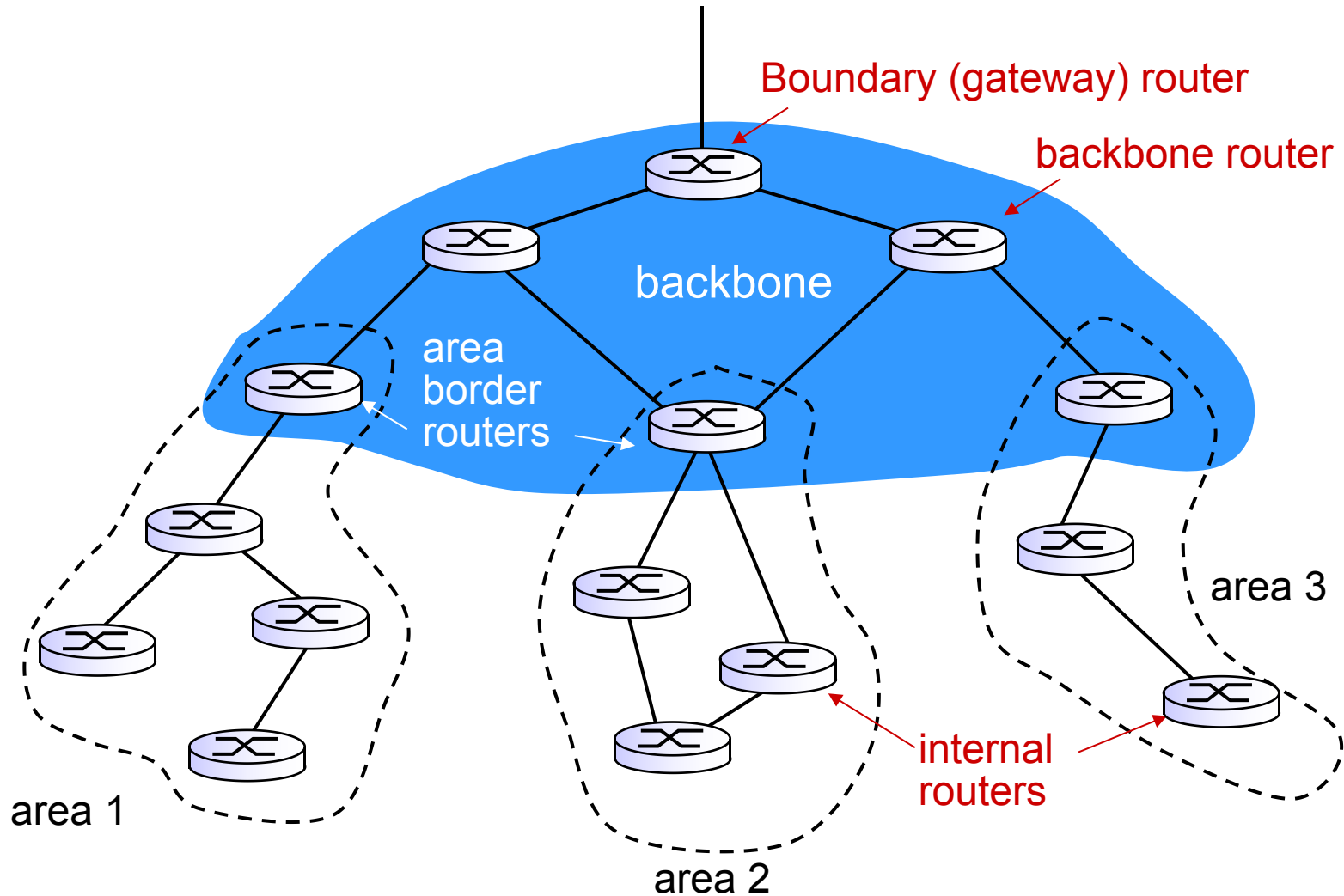
OSPF (Open Shortest Path First)

- “open”: publicly available
 - Message format, routing algorithms, link-state broadcast...
- uses link-state algorithm
 - link state packet dissemination
 - topology map at each node
 - route computation using Dijkstra's algorithm
- router floods OSPF link-state advertisements to all other routers in *entire* AS
 - carried in OSPF messages directly over IP (rather than TCP or UDP)
 - Reliable message transfer, link-state broadcast

OSPF “advanced” features

- **security**: all OSPF messages authenticated (to prevent malicious intrusion)
 - Password; private and public key
- **multiple** same-cost **paths** allowed (only one path in RIP)
- integrated uni- and **multi-cast** support:
 - Multicast OSPF (MOSPF) uses same topology data base as OSPF
- **hierarchical** OSPF in large domains.

Hierarchical OSPF



Hierarchical OSPF

- *two-level hierarchy*: local area, backbone.
 - link-state advertisements only in area
 - each nodes has detailed area topology; only know direction (shortest path) to nets in other areas.
- *area border routers*: routing packets outside the area.
- *backbone routers*: run OSPF routing limited to backbone.
- *Boundary (gateway) routers*: connect to other AS'es.

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

Internet inter-AS routing: BGP

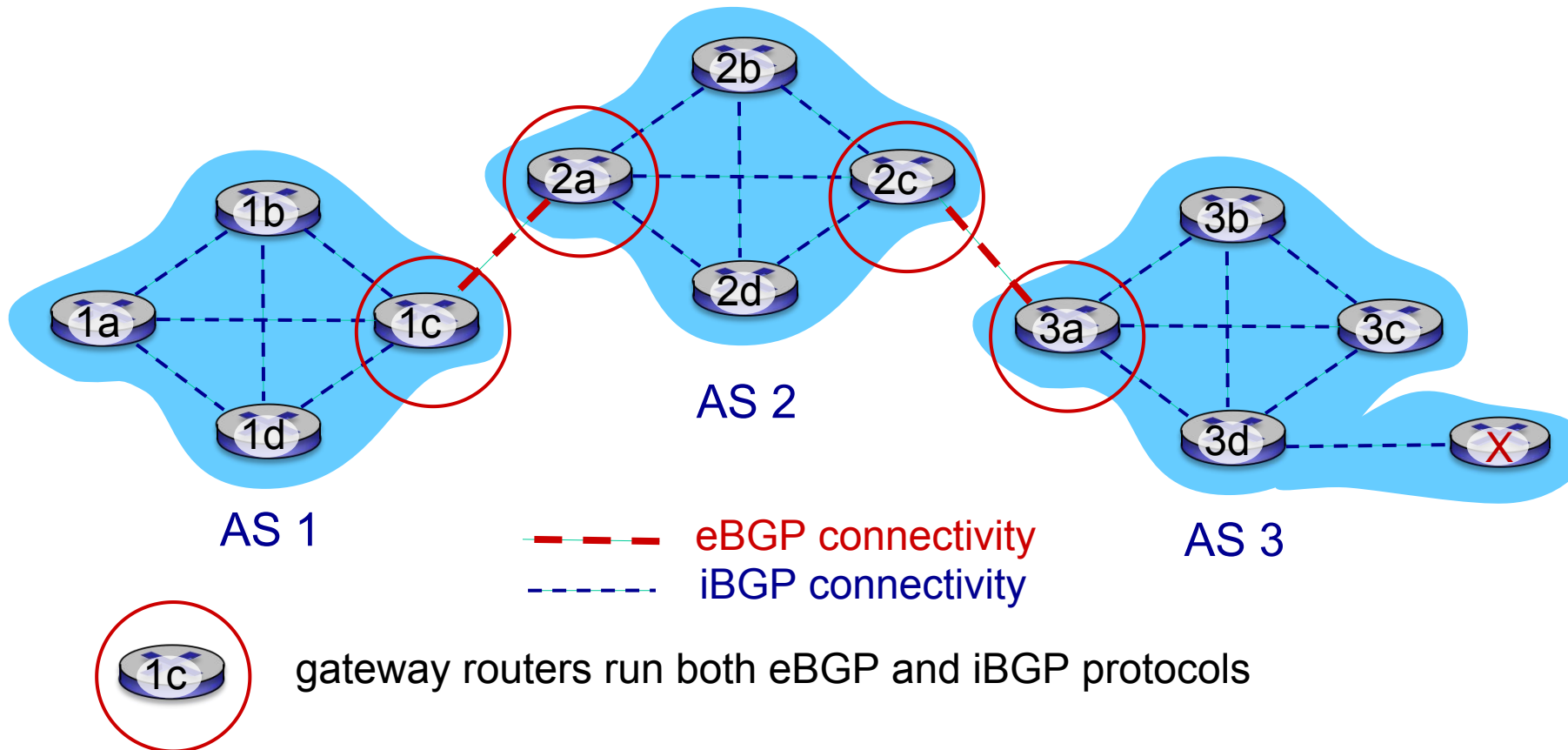
- **BGP (Border Gateway Protocol):** inter-domain routing protocol
 - “glue that holds the Internet together”
 - Decentralized, asynchronous, distance-vector
- Main functions BGP provides:
 - allows subnet to advertise its existence to rest of Internet: *“I am here”*
 - obtain subnet reachability information from neighboring ASes: **eBGP**
 - propagate reachability information to all AS-internal routers: **iBGP**
 - determine “good” routes to other networks based on reachability information and *policy*

Overview

- BGP: iBGP, eBGP
- Route Selection
- IP-Anycast
- BGP Routing Policy

BGP basics

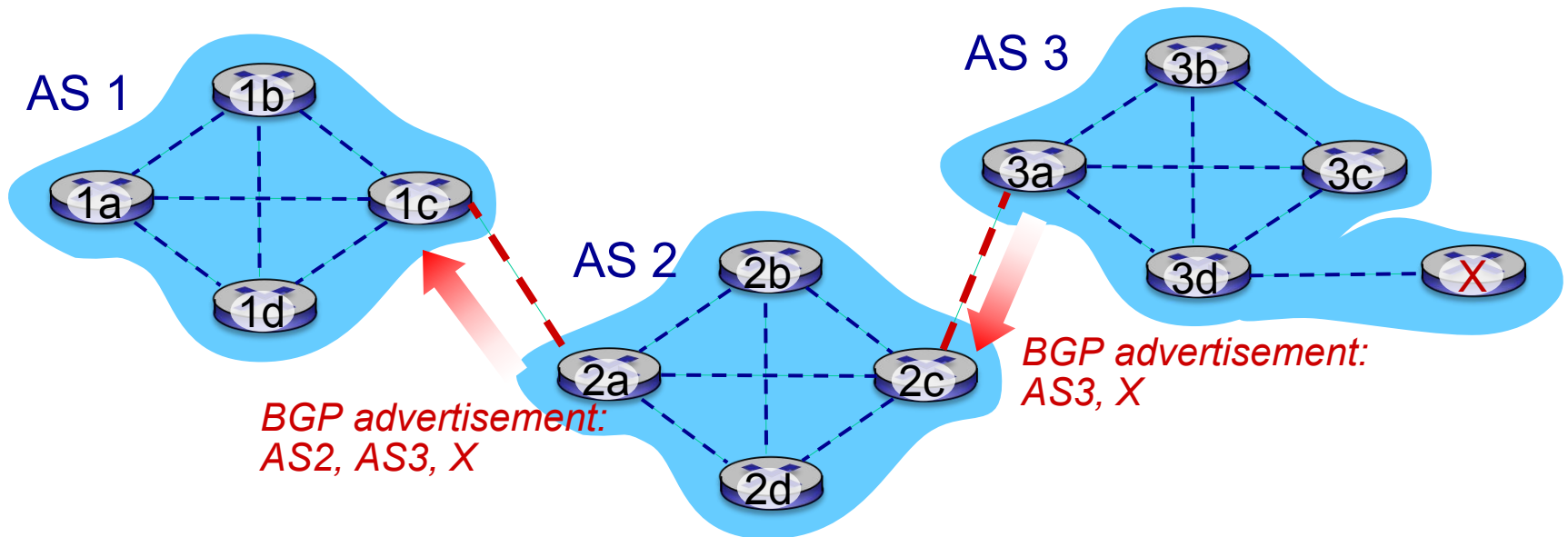
- Each pair of BGP routers (“peers”) exchanges BGP messages over TCP connection:
 - advertising *paths* to destination network prefixes (e.g., X)



eBGP basics

When AS3 gateway router 3a advertises path **AS3,X** to AS2 gateway router 2c:

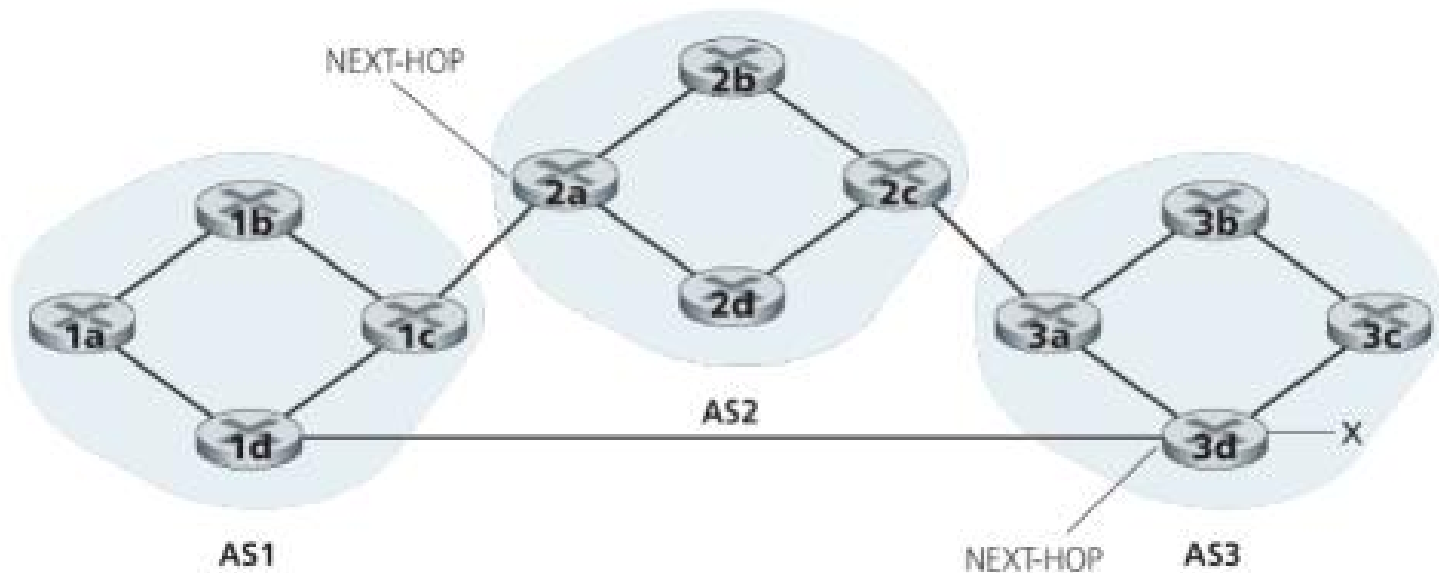
- AS3 *promises* to AS2 it will forward datagrams towards X



Path attributes and iBGP routes

- advertised prefix includes BGP attributes
 - Prefix (destination) + attributes = “route”
- two important attributes:
 - **AS-PATH**: list of ASes through which the advertisement has passed, e.g., AS2 AS3
 - Advertisement; prevent loops
 - **NEXT-HOP**: IP address of the router interface that **begins** the AS-PATH, e.g., IP of the interface of AS2 that begins AS2 AS3

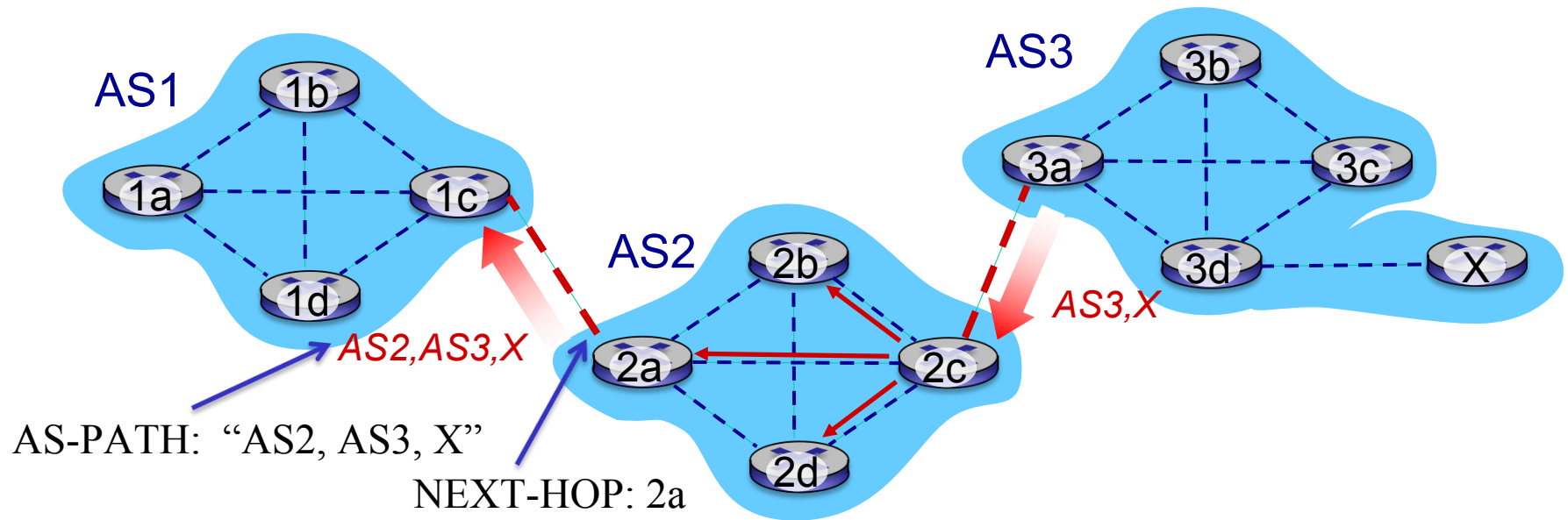
Path attributes and iBGP routes



IP address of leftmost interface for router 2a; AS2 AS3; x

IP address of leftmost interface of router 3d; AS3; x

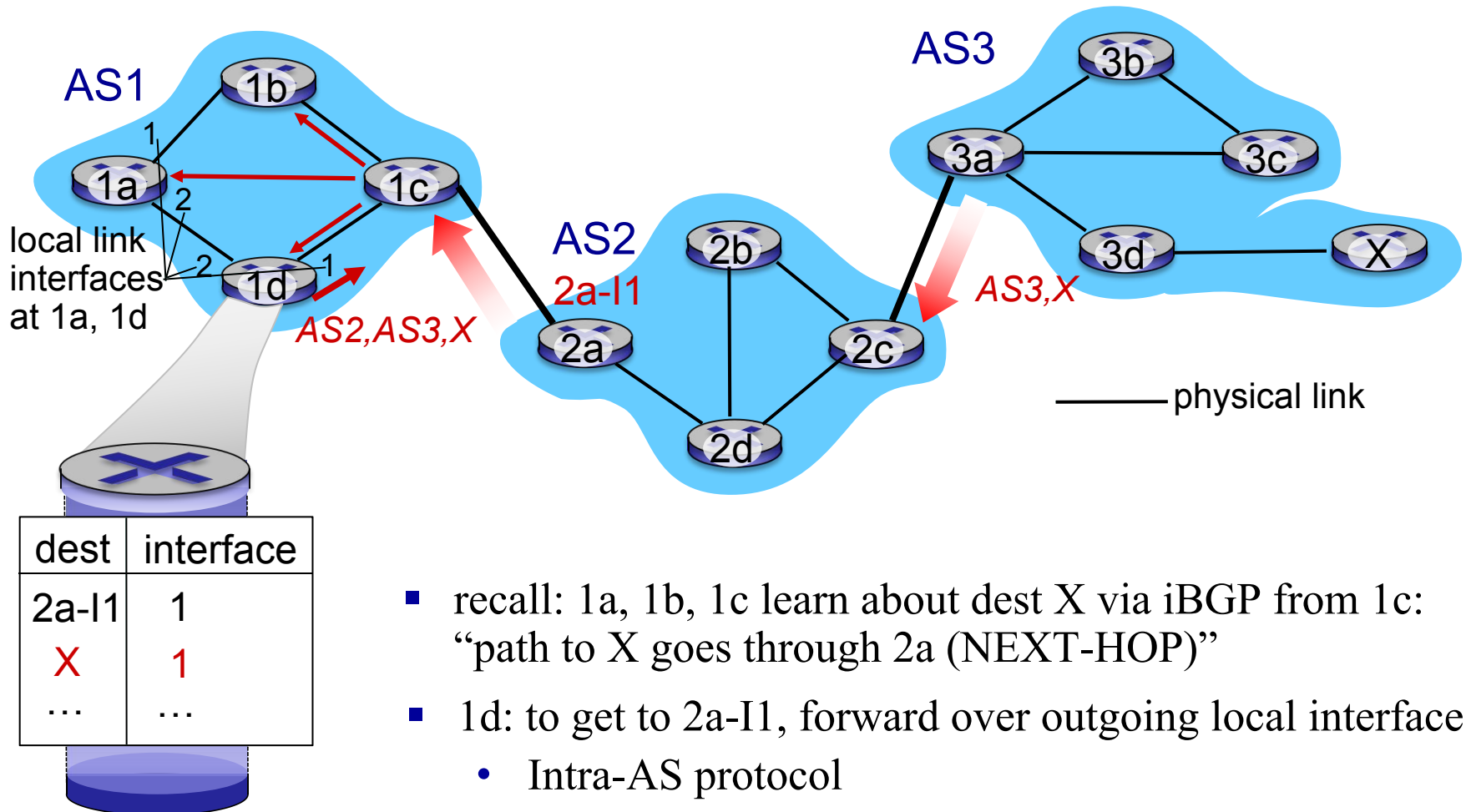
BGP path advertisement



- AS2 router 2c receives path advertisement **AS3,X** (via eBGP) from AS3 router 3a
- Based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers
- Based on AS2 policy, AS2 router 2a advertises (via eBGP) path **AS2, AS3, X** to AS1 router 1c

BGP, OSPF, forwarding table entries

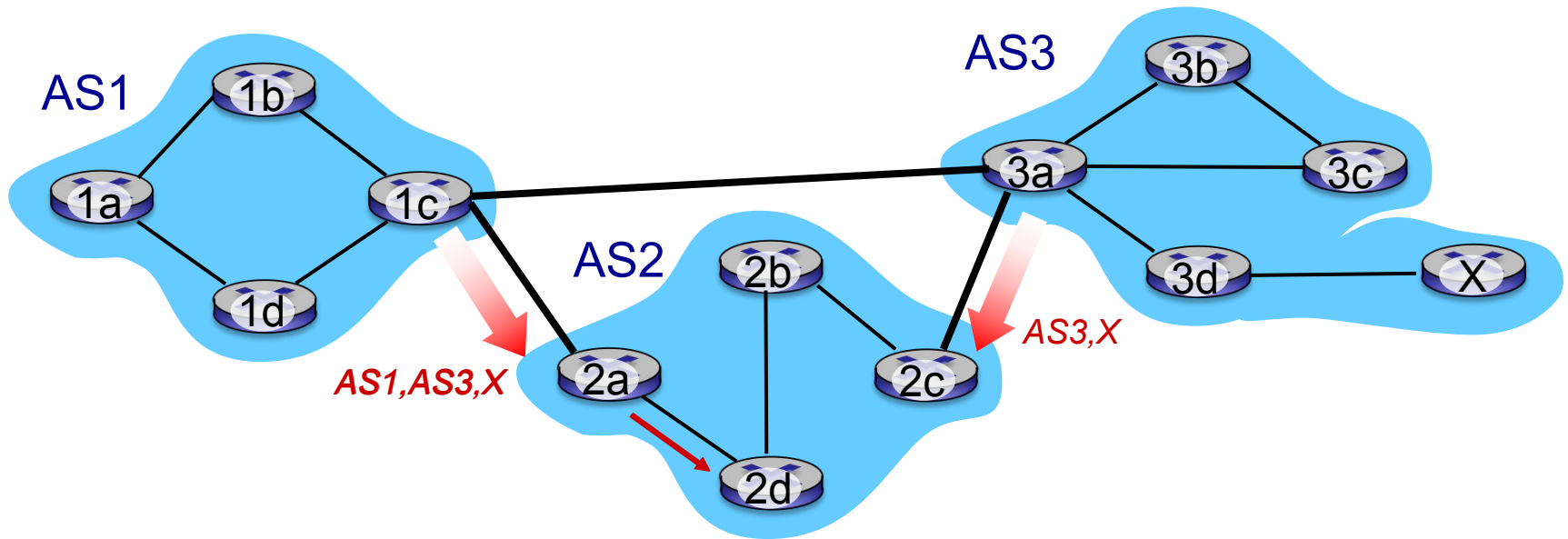
Q: how does router set forwarding table entry to distant prefix?



Overview

- BGP: iBGP, eBGP
- Route Selection
- IP-Anycast
- BGP Routing Policy

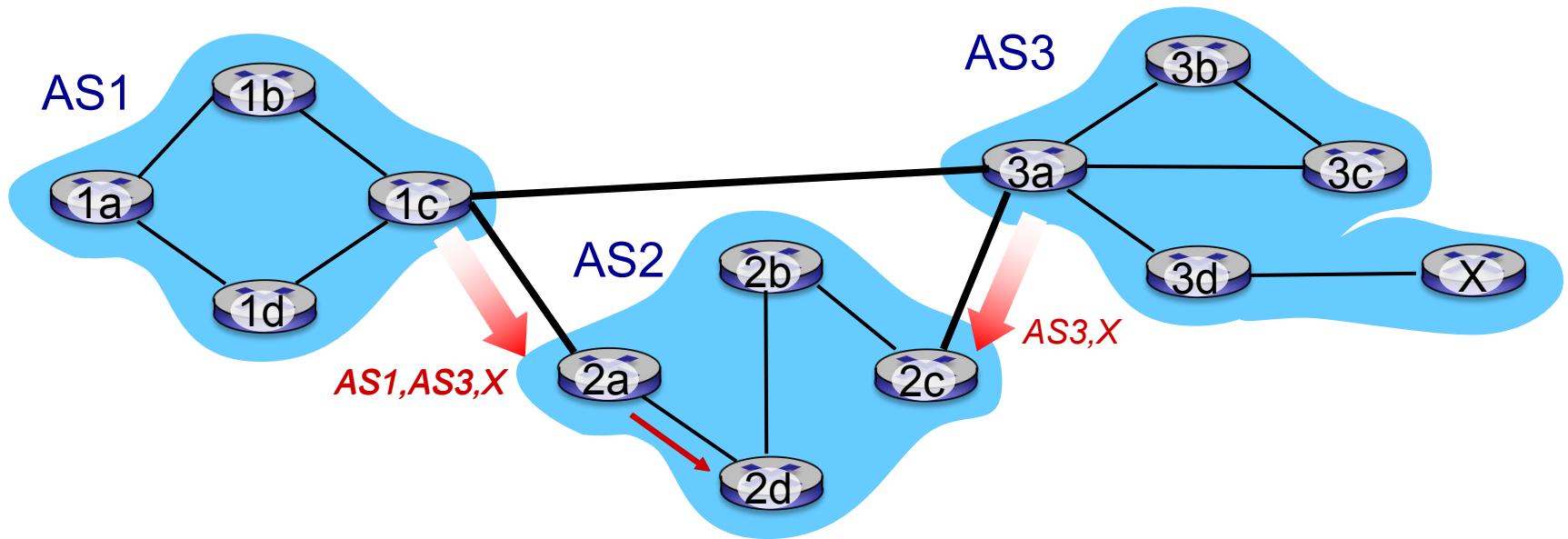
Route selection



A router may learn about **multiple** paths to destination:

- 2d learns path *AS1,AS3,X* from 1c
- 2d learns path *AS3,X* from 3a

Route selection: Hot Potato Routing



- 2d learns (via iBGP) it can route to X via 1c or 3a
- *hot potato routing*: choose local gateway that has least intra-domain cost (e.g., 2d chooses 2a, even though more AS hops to X): don't worry about inter-domain cost!

BGP route selection

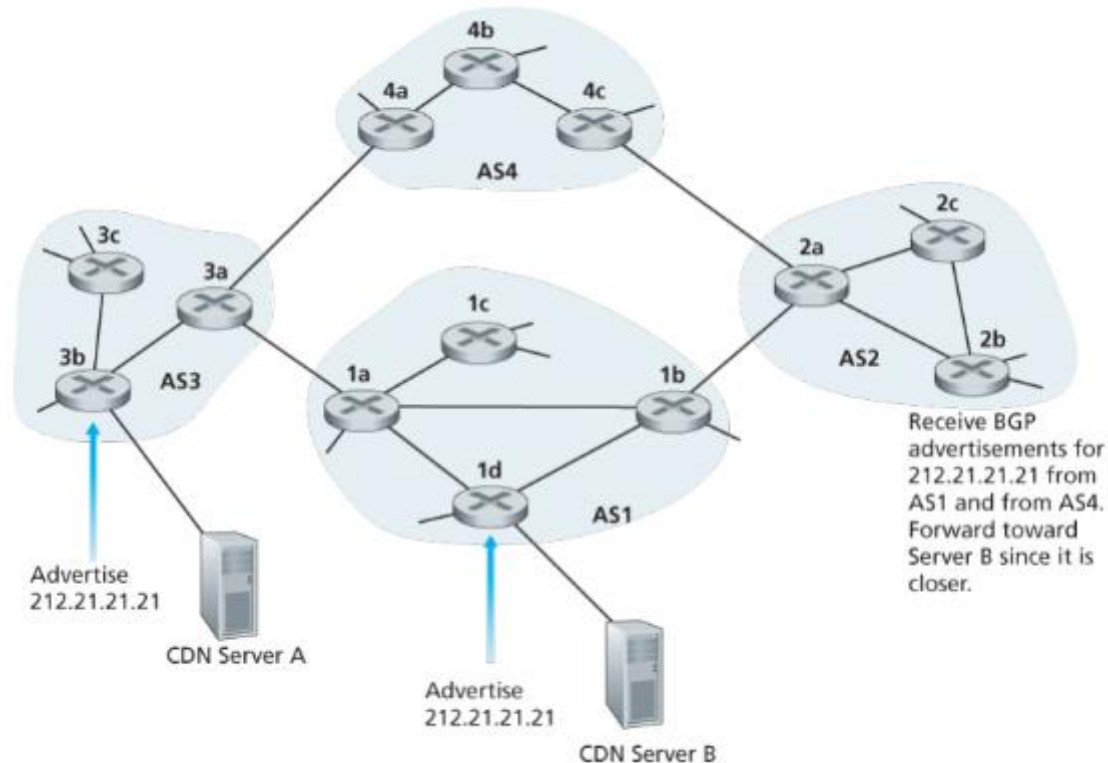
Router may learn about more than one route to destination AS, selects route based on:

1. local preference value attribute: policy decision
2. shortest AS-PATH
3. closest NEXT-HOP router: hot potato routing
4. additional criteria

Overview

- BGP: iBGP, eBGP
- Route Selection
- IP-Anycast
- BGP Routing Policy

IP-Anycast Service: CDN/DNS

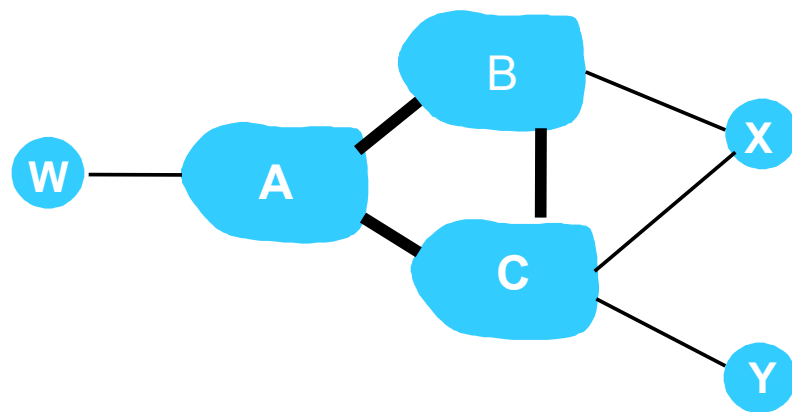


- CDN company assigns the **same IP address** to each server, and uses standard BGP to advertise this IP address from each server.
- When a BGP router receives multiple route advertisements for this IP address → **different paths to the same physical location**
- When configuring its routing table, each router will locally use the BGP route-selection algorithm to **pick the “best” route** to that IP address

Overview

- BGP: iBGP, eBGP
- Route Selection
- IP-Anycast
- **BGP Routing Policy**
 - determines whether to *advertise* path to other neighboring ASes

Routing Policy

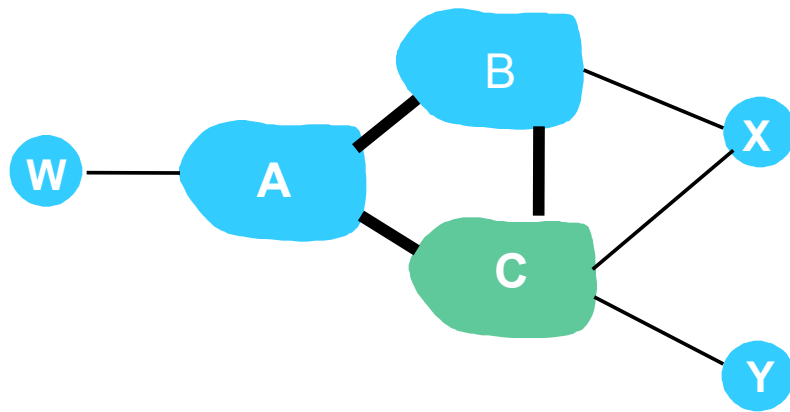


legend:  Backbone provider network
 customer network (ISP)

All traffic entering an ISP access network must be destined for that network, and all traffic leaving an ISP access network must have originated in that network.

- A,B,C are *provider networks*
- X,W,Y are customer (of provider networks)
- X is *dual-homed*: attached to two networks
- *policy to enforce*: X does not want to route from B to C via X
 - .. so X will not advertise to B a route to C
 - i.e., X has no paths to any other destinations except itself

Routing Policy



legend:  Backbone provider network
 customer network (ISP)

Suppose an ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs)

- A advertises path Aw to B and to C
- B advertises path BA_w to X
- B *chooses not to advertise* BA_w to C:
 - B gets no “revenue” for routing CBA_w, since none of C, A, w are B’s customers
 - C does not learn about CBA_w path
- C will route CA_w (not using B) to get to w

Why different Intra-, Inter-AS routing ?

policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its net.
- intra-AS: single admin, so no policy decisions needed

performance:

- intra-AS: can focus on performance
- inter-AS: policy may dominate over performance

scale:

- hierarchical routing saves table size, reduced update traffic

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP